MISSION DE LA SÉANCE

# Brancher l’IA. Garder le contrôle.

Un LLM derrière un port, puis un premier RAG capable de citer ses preuves et de s’abstenir.

Votre domaine · vos données

→ pour commencer



Photo Jessica Ruscello / Unsplash

LA CIBLE

# À la fin, votre assistant sait répondre… ou dire qu’il ne sait pas.

LLM

### Interchangeable

Fake et fournisseur live respectent le même contrat.

RAG

### Sourcé

Chaque réponse montre le passage réellement récupéré.

TESTS

### Sans réseau

Les scénarios automatiques restent déterministes.

SE REPÉRER AVANT DE CODER

# Le socle est fourni. Les preuves sont votre chantier.



Un document utile doit rester identifiable.

FOURNI

### Le chat et ADK fonctionnent déjà.

Lire `bootstrap.py`, le port de génération et son adaptateur. Commencer avec le profil fake.

À CRÉER

### Un petit corpus et son index.

Créer vos types de document/chunk, puis les use cases d’ingestion et de recherche.

À PROUVER

### Une citation… ou une abstention.

Conserver quatre questions et vérifier les sorties sans appel réseau dans les tests.

Photo Jessica Ruscello / Unsplash · Pictogrammes : tracés Lucide, licence ISC.

20 MINUTES MAXIMUM

# Préparez un corpus petit, propre et explicable.

1

## Choisir 3 à 8 documents

Markdown ou texte, publics ou synthétiques, liés à votre domaine.

2

## Donner des IDs stables

Une source doit rester identifiable après réindexation.

3

## Écrire 4 questions

Deux répondables, une ambiguë et une hors corpus.

### Bloqué sur les données ?

Utilisez temporairement `tp/fixtures/music-library/corpus/`. Ce jeu sert de secours, pas de sujet imposé.

```
uv sync --extra rag --group dev
```

LE RUNTIME EST FOURNI

# Utilisez ADK derrière le port fourni.




- ✓ Repérer TextGeneratorPort et son adaptateur ADK.
- ✓ Configurer Cohere ou Gemini dans .env.
- ✓ Échouer clairement si la config est inconnue.
- ✓ Tester le contrat avec le fake.

S2 · UTILISATION GUIDÉE

# Deux fournisseurs. Le même parcours fourni.

## Cohere

```
FILROUGE_LLM_PROVIDER=cohere
COHERE_MODEL=command-a-03-2025
COHERE_API_KEY=…
```

## Gemini

```
FILROUGE_LLM_PROVIDER=gemini
GEMINI_MODEL=gemini-2.5-flash
GOOGLE_API_KEY=…
```

Choisir un profil dans votre .env local, puis redémarrer Streamlit. Le profil fake reste disponible sans clé, avec une simulation explicite.

```
uv run adk web . --host 127.0.0.1 --port 8001
```

Choisir adk_agent pour observer le parcours. Les concepts ADK seront étudiés en S3. Le backend d’embedding reste un choix distinct.

USE CASE INGEST

# Transformez les documents en preuves recherchables.




01

## Idempotence

Réindexer la même version ne crée pas de doublon.

02

## Traçabilité

Chaque chunk garde son document, sa position et sa version.

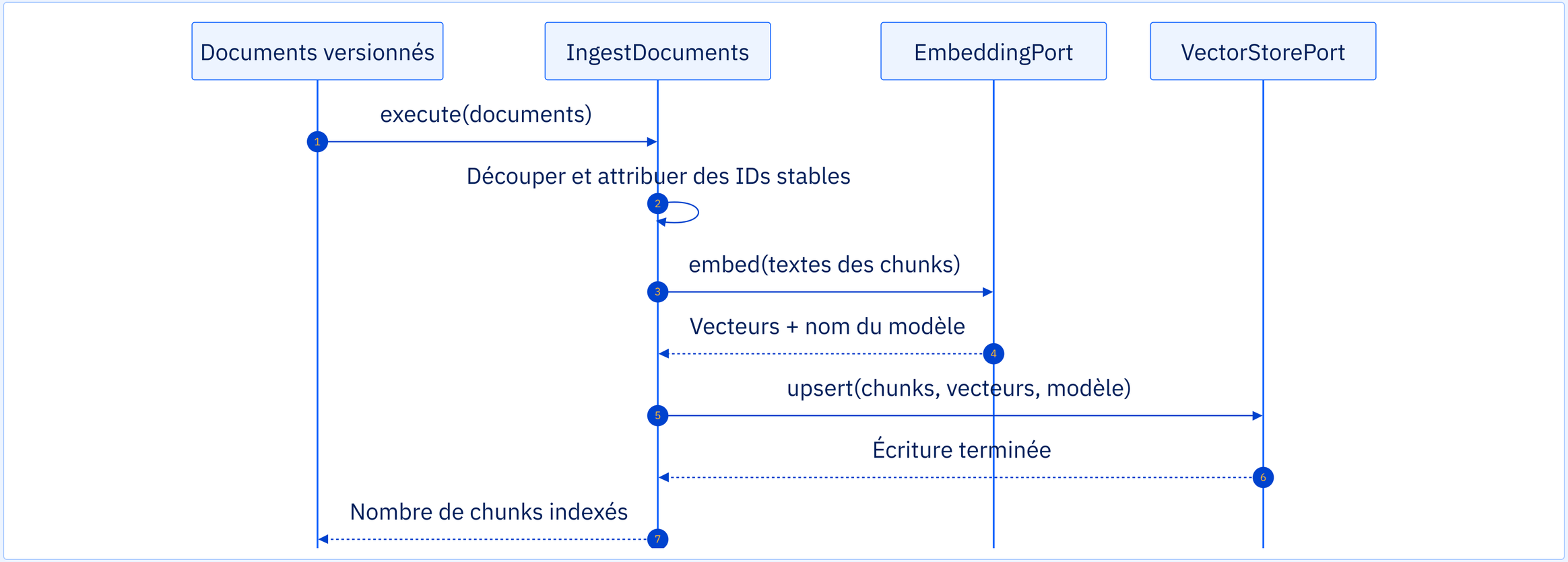
03

## Compatibilité

Un changement de modèle exige une réindexation explicite.

FLUX À IMPLÉMENTER · SÉQUENCE

# Indexer, c’est garder le lien avec le document.




Les IDs associent document, version et position. Le même modèle produit les vecteurs du corpus et des questions. Flèche pleine : appel ; pointillés : retour.

ATELIER · INGESTION, PUIS SEULEMENT PERSISTANCE

# Commencez par un document et un faux stockage.

01 **Définir les données minimales**

Dans `domain/retrieval.py`, créer Document et Chunk : ID, texte, source, version et position.

02 **Écrire le test avant l'adaptateur**

Dans `tests/test_ingestion.py`, ingérer deux fois le même document. Le fake remplace les IDs existants.

03 **Raccorder les ports**

Créer `IngestDocuments` dans les use cases ; injecter embedding et vector store. Puis brancher Chroma au bootstrap.

À CRÉER · tests/test_ingestion.py · squelette à adapter

```
doc = Document(
    id="guide", title="Guide",
    text="Un premier paragraphe.",
    version="v1",
)
ingest.execute([doc])
ingest.execute([doc])
# À compléter : vérifier le nombre
# de chunks et leur provenance.
```

**Vérifier.** Le second passage ne double pas les chunks. Si le compte augmente, examiner les IDs et la logique d'upsert.

Étapes détaillées et commandes copiables dans le README du TP.

USE CASE ASKWITHRAG

# Recherchez d’abord. Générez ensuite.

1

## Valider la question

Taille, vide, caractères et contexte autorisé.

2

## Récupérer 1 à 3 passages

Seuil minimal et nombre de sources bornés.

3

## Construire le prompt

Question + passages numérotés + consigne de citation.

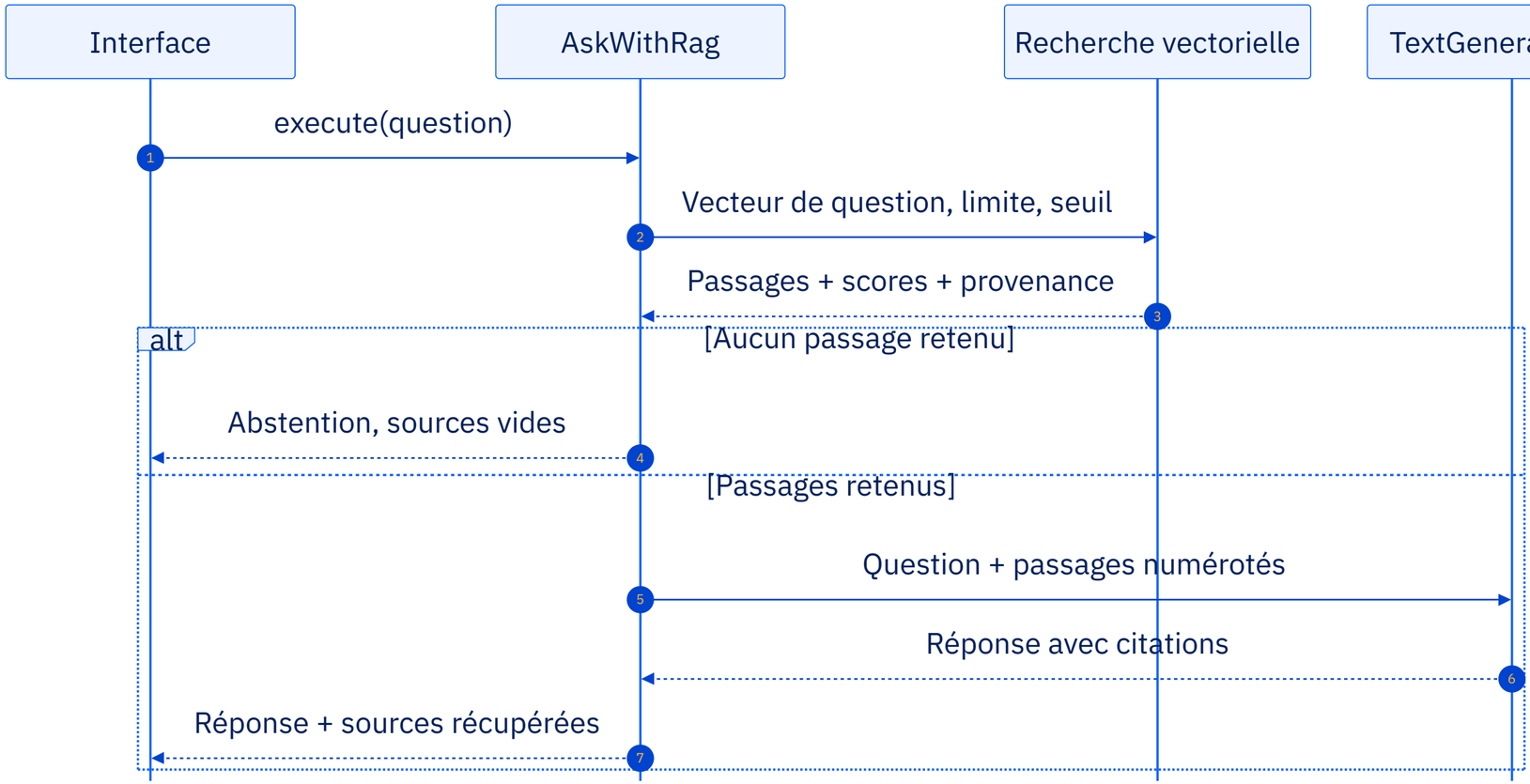
### La règle d’abstention

Si aucun passage ne dépasse le seuil, ne pas appeler le LLM. Retourner une réponse explicite avec zéro source.

```
« Je ne dispose pas de preuve suffisante dans le corpus. »
```

FLUX À IMPLÉMENTER · SÉQUENCE

# Sans passage pertinent, aucun appel au LLM.




Récupérer au plus trois passages, filtrer par score, puis générer avec les preuves retenues. Flèche pleine : appel ; pointillés : retour.

Lire de haut en bas · les noms de ports désignent des contrats de l'application.

ATELIER · ASKWITHRAG

# Testez le refus avant la première belle réponse.

### 01 Préparer un générateur espion

Un fake compte les appels à `generate_reply`. Un faux store retourne d’abord une liste vide.

### 02 Écrire deux scénarios

Zéro passage : abstention et zéro appel. Passage pertinent : réponse et source copiée du résultat de recherche.

### 03 Brancher l’interface en dernier

Afficher réponse, source et extrait. Si un score surprend, inspecter les passages avant de modifier le prompt.

À CRÉER · tests/test_rag.py · assertions à compléter

```
reply = ask.execute("Question hors corpus")

assert reply.is_abstention
assert reply.sources == []
assert generator.calls == 0

# Puis ajouter le cas avec une preuve.
# Le fake d’embedding vérifie le flux ;
# il ne mesure pas la qualité sémantique.
```

**Vérifier.** Une source affichée existe réellement dans les passages récupérés. Le fournisseur de génération et celui d’embedding sont des choix distincts.

Étapes détaillées et commandes copiables dans le README du TP.

CAPSULE C4 · SRP

# Une fonction pure pour la salutation. Rien de plus.

## À extraire

`build_greeting(name, language)` construit le texte. Streamlit lit les champs et affiche le résultat.

- aucun import Streamlit dans la fonction ;
- langues supportées testées ;
- entrée invalide explicite.

```
uv run pytest
uv run ruff check .
```

- ✓ Fake et live ont le même contrat.
- ✓ Aucun test n’appelle le réseau.
- ✓ La salutation est testée directement.

MONTREZ, NE RACONTEZ PAS

# Votre démonstration tient en quatre questions.

01

### Réponse directe

Une information présente dans un seul document.

02

### Deux sources

Une réponse qui oblige à montrer deux passages.

03–04

### Limite

Une question ambiguë puis une question hors corpus.

```
git add .
git commit -m "Ajoute le LLM derrière un port et le premier RAG"
```

AVANT DE QUITTER S2

# Votre système est remplaçable, sourcé et testable.

## Vous avez construit

- un port LLM et plusieurs adaptateurs ;
- une ingestion idempotente ;
- un RAG avec citations ;
- une abstention explicite.

## À conserver pour S3

- le fake pour les tests ;
- les quatre questions de référence ;
- les IDs et versions du corpus ;
- les erreurs de configuration visibles.

```
uv run pytest  &&  uv run ruff check .
```

S2 · CONSIGNES À GARDER PROJETÉES

# À vous : un RAG qui montre ses preuves.

01 **Préparer le corpus**

3 à 8 documents identifiés et versionnés ; écrire les quatre questions de référence.

02 **Lire le chat fourni**

Garder ADK derrière TextGeneratorPort ; tester fake, puis le profil live choisi si configuré.

03 **Construire ingestion et recherche**

Commencer avec des fakes ; IDs stables, upsert, trois passages maximum et seuil explicite.

04 **Prouver réponse et abstention**

Afficher les sources ; sans passage, vérifier zéro appel au modèle. Extraire la salutation pure.

## À montrer avant de terminer

- Une question répondue avec son extrait.
- Une question hors corpus refusée sans génération.
- Réindexation sans doublon et tests sans réseau.

```
uv run pytest
uv run ruff check .
```

Bloqué ? README de ce TP → « Pas à pas ». Corpus de secours : `tp/fixtures/music-library/corpus/`.